

Chuẩn hoá giao diện trong thời đại AI: quy trình DS → SD → Thiết kế → Code còn hiệu quả không?

Đánh giá quy trình 4 bước chuẩn hoá giao diện khi có AI tham gia tự động hoá từng khâu – tổng hợp từ blog chính thức của Figma, Brad Frost, chuẩn W3C DTCG, hệ sinh thái shadcn/v0, Storybook, báo cáo AI in Design 2026 và phản biện từ Linear.

Ngày: 14 / 07 / 2026 Nguồn tham chiếu: 18

Góc nhìn: Chuyên gia thiết kế & kiến trúc FE ~20 năm

MỤC LỤC

- Tóm tắt điều hành & quy trình được đánh giá
- Đánh giá tính phù hợp của từng bước
- Quy trình nâng cấp cho thời đại AI
- Quy trình cộng đồng đề xuất: "Design System as AI Context"
- Bốn khuyến nghị thực chiến
- Tổ chức thực thi cho mô hình hai phòng
- Phụ lục: danh mục nguồn

01 Tóm tắt điều hành & quy trình được đánh giá

Kết luận chính: quy trình 4 bước *đúng về nguyên lý (~80%)* nhưng sai ở hai giả định nền tảng — nó là quy trình **một chiều (waterfall)** và ngầm định **Figma là**

source of truth. Cả hai giả định này đang bị cộng đồng đảo ngược trong 2 năm qua: nguồn chân lý dịch chuyển về *tokens + code*, còn Figma trở thành môi trường khám phá và thao tác. AI không thay thế bước nào — nó thay thế *chất keo dịch tay* giữa các bước, với điều kiện phải bổ sung vòng phản hồi và cổng kiểm chứng tự động.

QUY TRÌNH ĐƯỢC ĐÁNH GIÁ (NGUYÊN TRẠNG)

Bước 1. Dựng Design System (DS) trên Figma
quy định mọi tiêu chuẩn thiết kế



Bước 2. Biến DS thành System Design (SD) trên các framework FE
desktop lẫn mobile



Bước 3. Thiết kế màn hình chi tiết theo nghiệp vụ trên Figma
dựa trên DS



Bước 4. AI chuyển thiết kế màn hình thành code FE
dùng SD đã code làm nền tảng



⚠ Không có đường quay về – mọi chỉnh sửa ở bước 4 làm DS Figma và SD code trôi dạt (drift)

BƯỚC	PHÁN QUYẾT	LÝ DO CỐT LỖI
1. DS trên Figma	Vấn đúng	Nhưng tiêu chuẩn “đạt” đổi từ <i>người đọc được</i> sang <i>máy đọc được</i> (variables, auto-layout, naming = props).
2. DS → SD code	Đúng nhất	Coded components là thứ AI cần để không “bịa” UI – nhưng chiều mũi tên đang đảo: cộng đồng chuyển sang code-first.

BƯỚC	PHÁN QUYẾT	LÝ DO CỐT LỖI
3. Hi-fi mọi màn hình	Lung lay	Kinh tế học đã đổi: màn hình pattern chuẩn (60–80% hệ nghiệp vụ) không cần hi-fi nữa; chỉ giữ hi-fi cho flow mới.
4. AI design-to-code	Đúng hướng, sai cơ chế	Nếu “chuyển đổi” = tái tạo pixel thì thất bại; cách đúng là structured context qua MCP + Code Connect.
Cấu trúc tổng thể	Thiếu 3 thứ	Một source of truth duy nhất · vòng phản hồi (feedback loop) · cổng kiểm chứng máy (verification gates).

02 Đánh giá tính phù hợp của từng bước

2.1 • Bước 1 – Dựng DS trên Figma Vẫn đúng, tiêu chuẩn “đạt” đã thay đổi

DS trên Figma vẫn là nền móng, và AI làm nó *quan trọng hơn chứ không kém đi*. Brad Frost — cha đẻ Atomic Design — lập luận rằng AI là “người dùng lý tưởng” của một design system được tài liệu hoá tốt; ngược lại, AI không có DS chỉ sinh ra giao diện generic.

✕ **TRÍCH NGUỒN** — Brad Frost, “Design systems in the time of AI”

Design system sinh ra để giải phóng con người khỏi những việc lặp đi lặp lại — như Josh Clark nói, nó “gánh giùm phần việc nhàm chán, để designer và developer không phải làm”. Khi AI được huấn luyện trên một thư viện component có props rõ ràng và tài liệu TypeScript đầy đủ, nó có thể dựng giao diện chỉ qua vài câu hội thoại — rút quãng đường từ bảng trắng đến prototype từ vài phút xuống vài giây. Dù vậy, những quyết định đòi hỏi sự tinh tế — nên làm component cứng hay linh hoạt, phân tầng hệ thống ra sao, cái gì đáng là component còn cái gì chỉ là công thức lắp ghép (recipe) — vẫn phải do con người phán đoán.

“The design system carries the burden of the boring, so that designers and developers don’t have to.”
— Josh Clark

bradfrost.com → [Design systems in the time of AI](#)

Điểm thay đổi then chốt: tiêu chuẩn “DS tốt” giờ được đo bằng **máy đọc được hay không**:

TRƯỚC ĐÂY (NGƯỜI ĐỌC)	BÂY GIỜ (MÁY + NGƯỜI ĐỌC)
Style guide đẹp, trang cover chỉn chu	Variables/tokens theo chuẩn, không hardcode màu
Component có variant	Component property đặt tên trùng props trong code (<code>variant</code> , <code>size</code> , <code>disabled</code>)
Layer name tùy hứng	Naming nhất quán, auto-layout 100%, constraints đúng
Mô tả hành vi bằng lời khi hộp	Annotation ngay trong file về hover / responsive / state

<

TRÍCH NGUỒN

– LogRocket, hướng dẫn cấu trúc file Figma cho MCP

Những thói quen tưởng nhỏ — đặt tên layer cẩn thận, dùng component nhất quán, bật auto-layout, gán mọi giá trị chữ / màu / khoảng cách vào biến (variables) — giờ chính là nền móng quyết định việc coding agent có dựng lại thiết kế chính xác hay không. Chúng **không còn là thứ “có thì tốt”, mà là điều kiện bắt buộc** của quy trình.

[blog.logrocket.com → How to structure Figma files for MCP and AI-powered code generation](https://blog.logrocket.com/how-to-structure-figma-files-for-mcp-and-ai-powered-code-generation/)

2.2 • Bước 2 – Biến DS thành SD code

Đúng, nhưng chiều mũi tên đang bị đảo

Đây là bước đúng đắn nhất trong quy trình: **thư viện component đã code chính là thứ AI cần để không “bịa” UI**. Tuy nhiên có hai điều chỉnh lớn.

(a) Cộng đồng đang chuyển sang code-first. Hiện tượng đáng chú ý nhất 2025–2026 là shadcn/ui — design system do developer xây, công khai mã nguồn để LLM đọc được, và trở thành “ngôn ngữ mặc định” của mọi công cụ AI sinh UI:

× TRÍCH NGUỒN — Pawel Klasa, Design Bootcamp (Medium)

Cứ thử nhờ Claude, Cursor, vò hay Lovable dựng một giao diện mà xem: kết quả gần như lúc nào cũng **mang dáng dấp shaden** — từ component, pattern cho đến cách lắp ghép. Các công cụ AI đã âm thầm chuẩn hoá quanh nó. Design system quan trọng nhất năm 2026 hoá ra do một developer xây — còn giới designer thì... bỏ lỡ.

medium.com → [The most important Design System in 2026 that designers missed](#)

× TRÍCH NGUỒN — shadcn/ui docs & Vercel

shadcn/ui công khai toàn bộ mã nguồn component để LLM có thể đọc, hiểu và chỉnh sửa. Các team có thể dựng một **registry nội bộ** — nơi phân phối component, token và quy ước của riêng mình — để mọi dự án cùng kéo về từ một nguồn chân lý duy nhất. Một registry có thể chứa từ hooks, tiện ích, design token, font, cho đến nguyên cả một design system, template, codemod — **và thậm chí cả bộ chỉ dẫn dành riêng cho AI agent**.

ui.shadcn.com → [Introduction](#) · vercel.com → [What is shadcn/ui?](#)

(b) Việc “biến” không còn là chép tay, mà là pipeline tự động qua tokens.

Chuẩn W3C DTCG đã ra bản stable — lớp “nguyên tử” của DS giờ sync bằng CI/CD chứ không phải bằng mắt:

× TRÍCH NGUỒN — W3C Design Tokens Community Group, 28/10/2025

Đặc tả Design Tokens vừa đạt bản ổn định đầu tiên (phiên bản 2025.10): một định dạng chung, không lệ thuộc nhà cung cấp nào, đủ chín để dùng trong production khi cần chia sẻ các quyết định thiết kế giữa những công cụ và nền tảng khác nhau. Độ tin cậy của chuẩn được bảo chứng bằng bản hiện thực tham chiếu trong **Style Dictionary, Tokens Studio và Terrazzo**; hơn 10 công cụ khác — từ Figma, Sketch, Penpot, Framer đến Knapsack, Supernova, zeroheight — đã hỗ trợ hoặc đang triển khai.

w3.org → [Design Tokens specification reaches first stable version](#)

2.3 • Bước 3 – Thiết kế chi tiết mọi màn hình trên Figma

Bước lung lay nhất

Kinh tế học của bước này đã thay đổi hẳn:

× TRÍCH NGUỒN — Designer Fund, AI in Design Report 2026 (chương Craft)

91% designer nay dùng AI hằng tuần — năm ngoái con số này mới là 54%. **Một nửa trong số họ đã đưa code do AI viết lên production**, chính thức khép lại cuộc tranh cãi “designer có nên biết code hay không”. Có người giờ làm toàn bộ khâu khám phá ý tưởng bằng prototype code thay vì trên canvas; có người vẫn giữ mockup Figma, nhưng coi prototype bằng code là một phần tự nhiên của quy trình.

stateofaidesign.com → Craft

Khuyến nghị thực chiến: **không bỏ bước 3, nhưng phân tầng fidelity theo độ mới của bài toán:**

- **Màn hình chuẩn hoá cao** (CRUD, danh sách, form, admin — chiếm 60–80% một hệ nghiệp vụ lớn): *không cần hi-fi Figma nữa*. Wireframe + spec nghiệp vụ + AI compose từ SD là đủ; review trên bản chạy thật.
- **Màn hình mới về pattern hoặc quyết định trải nghiệm lớn** (dashboard mới, luồng phức tạp): vẫn design hi-fi trên Figma — vì nhảy vào code quá sớm là đưa constraint vào quá sớm.

× TRÍCH NGUỒN — Karri Saarinen, CEO Linear (phản biện quan trọng nhất)

Với Saarinen, design về bản chất là **một cuộc tìm kiếm (search)**, không phải một dây chuyền sản xuất. Ông cảnh báo rằng các công cụ chạy theo năng suất có thể bóp nghẹt chính khoảng không gian khám phá mà design cần: “Mỗi khi designer thiên về vai người xây dựng, một phần lý tưởng và sức sáng tạo sẽ chết đi. Không phải vì việc xây dựng là xấu, mà vì bạn đưa các ràng buộc vào quá sớm — sớm hơn thời điểm lẽ ra chúng mới nên xuất hiện.”

“Whenever a designer becomes more of a builder, some idealism and creativity dies. Not because building is bad, but because you start out including constraints earlier in the process than they should.”

ai-creators.tech → Design Is Search

× TRÍCH NGUỒN – Karri Saarinen trên X: chính Linear cũng làm hybrid

Đợt làm mới giao diện của Linear khởi đầu bằng một vòng khám phá thiết kế trên **Figma**; sau đó hai người trong team dựng prototype và công cụ **bằng AI agent**; rồi họ **quay lại Figma** chỉnh tiếp, viết thêm code, và cuối cùng phát hành nội bộ sau một *feature flag*. Nói cách khác: ngay cả đội ngũ nổi tiếng khắt khe về “craft” cũng đi theo vòng lặp hai chiều Figma ↔ code, chứ không chọn phe nào.

x.com/karrisaarinen → [Little big update on the Linear UI](#)

× TRÍCH NGUỒN – Department of Product, phòng văn CEO Figma

Trước câu hỏi “thiết kế hay code”, CEO của Figma cho rằng đây là một **thế lưỡng phân giả (false dichotomy)** — bản thân cách đặt câu hỏi đã sai. Hướng đi của công ty là một nền tảng liền mạch, nơi người ta vừa thiết kế tự do, vừa prototype bằng code, vừa ship thẳng ra production.

departmentofproduct.substack.com → [Figma’s CEO says the Design vs Code debate is a “false dichotomy”](#)

2.4 • Bước 4 – AI chuyển thiết kế thành code

Đúng hướng, cơ chế phải đổi hoàn toàn

Đây là chỗ dễ làm sai nhất. Nếu “chuyển đổi” nghĩa là xuất hình ảnh → AI tái tạo pixel, nó sẽ thất bại có hệ thống:

× TRÍCH NGUỒN – Banani, AI Design-to-Code Tools Guide

Khoảnh khắc bạn export thiết kế ra khỏi Figma, **khoảng 80% metadata — component, auto-layout, constraints, design token — bốc hơi**. Thứ công cụ AI nhận vào chỉ còn là hình ảnh, và nó buộc phải đoán mò ý đồ của người thiết kế.

banani.co → [AI Design-to-Code Tools: The Complete Guide](#)

× TRÍCH NGUỒN – Shahid Pattani, AI Monks (Medium)

Vấn đề của code do AI sinh từ thiết kế là nó **“trông thì đúng, nhưng không đúng”**: test vẫn xanh với những thứ chắc chắn sẽ hỏng ở production; breakpoint responsive được “xử lý” nhưng quên mất vùng chạm (touch target) trên mobile; các tính năng accessibility được “thêm vào” nhưng thực chất chỉ là ảo giác của model — có tên gọi mà không có hiện thực.

[medium.com](https://medium.com/@shahidpattani/design-to-code-ai-is-not-magic-here-s-why-it-fails-sometimes) → [Design-to-Code AI Is Not Magic – Here’s Why It Fails Sometimes](https://medium.com/@shahidpattani/design-to-code-ai-is-not-magic-here-s-why-it-fails-sometimes)

Cách đúng năm 2026 là **structured context, không phải pixel**: agent đọc thiết kế qua Figma MCP server (lấy layout, tokens, cấu trúc component dạng dữ liệu), và nhờ **Code Connect** biết chính xác node Figma này ứng với component nào trong repo — nên nó gọi *component* thật thay vì tái tạo.

× TRÍCH NGUỒN – Figma Blog (chính thức), câu quote đắt nhất của cả chủ đề

Bắt một AI agent viết code mà không cho nó biết gì về design system, chẳng khác nào bắt một kỹ sư mới vào công ty phải ship code ngay ngày đầu tiên — trước cả khi được onboarding.

“Asking an AI agent to generate code without design system context is like asking a new engineer to start shipping code before onboarding.”

[figma.com/blog](https://figma.com/blog/design-systems-and-ai-why-mcp-servers-are-the-unlock) → [Design Systems And AI: Why MCP Servers Are The Unlock](https://figma.com/blog/design-systems-and-ai-why-mcp-servers-are-the-unlock)

2.5 • Ba lỗi hổng cấu trúc của quy trình

1. **Double source of truth**: DS Figma (bước 1) và SD code (bước 2) là hai bản thể song song. Không có cơ chế sync tự động thì sau 6 tháng chúng khác nhau, và AI ở bước 4 nhận context mâu thuẫn.
2. **Không có feedback loop**: sửa trong code không quay về Figma; bài học từ correction không quay về quy trình.
3. **Không có verification gate**: thiếu vòng kiểm chứng tự động (visual diff, token lint, a11y) thì thời gian review nuốt hết thời gian tiết kiệm được.

× TRÍCH NGUỒN — CIO.com, khảo sát niềm tin của developer

Code do AI viết thường lọt qua các vòng review truyền thống vì cú pháp trông sạch sẽ, nhưng lại hay **trượt ở độ khớp kiến trúc**: đưa vào những pattern đi ngược quy ước sẵn có, hoặc tạo ra sự phụ thuộc ngầm giữa các service. Hệ quả là thời gian kiểm tra và sửa lại có thể **ăn sạch phần năng suất tưởng như tiết kiệm được** — thậm chí có lúc còn lâu hơn tự viết từ đầu.

[cio.com](#) → [Developers still don't trust AI-generated code](#)

03 Quy trình nâng cấp cho thời đại AI

Nguyên tắc thiết kế lại: một source of truth duy nhất là tokens + code; Figma là môi trường thao tác và khám phá; mọi thứ có đường về (round-trip); mọi output của AI phải qua cổng kiểm chứng.

KIẾN TRÚC QUY TRÌNH V2 – CÓ SYNC, CÓ GATE, CÓ VÒNG HỌC

TOKENS – DTCG JSON, sống trong git

Source of truth #1 · Style Dictionary chạy trong CI → CSS vars / Tailwind / TS types / Swift / Kotlin

▲▼ sync 2 chiều (Tokens Studio / Figma Variables native DTCG)

DS trên Figma

components · auto-layout · naming = props · annotations · nơi exploration

SD code (React/Compose/SwiftUI)

Source of truth #2 · Storybook + tests + registry kiểu shadow

◀ Code Connect map: node Figma ↔ component thật ▶

▼ thiết kế màn hình: hi-fi cho flow mới · lo-fi/skip cho CRUD

AI AGENT (Claude Code / Cursor)

context vào: Figma MCP (layout + tokens của màn hình) · Storybook MCP (props, stories, usage thật) · CLAUDE.md rules (quy ước, cấm hardcode)



VERIFICATION GATES (tự động, CI + local)

Gate A: build/typecheck/test · Gate B: token lint (0 hardcode) · Gate C: screenshot ↔ design diff · Gate D: a11y scan (axe)



PR review (con người) → merge

🔄 FEEDBACK LOOP: learnings/corrections → cập nhật rules, tokens, Code Connect, stories

3.1 · Nâng cấp bước 1+2: hợp nhất bằng token pipeline

Token định nghĩa một lần dạng DTCG, generate ra cả hai phía — hết drift ở tầng nguyên tử:

TOKENS/COLOR.TOKENS.JSON – CHUẨN W3C DTCG (STABLE 2025.10)

```
{
  "color": {
    "brand": {
      "primary": { "$type": "color", "$value": "#0B5FFF" }
    },
    "action": {
      "primary-bg": {
        "$type": "color",
```

```

    "$value": "{color.brand.primary}",
    "$description": "Background for primary buttons and CTAs"
  }
},
"spacing": {
  "md": { "$type": "dimension", "$value": "16px" }
}
}

```

STYLE-DICTIONARY.CONFIG.JS – CHẠY TRONG CI MỖI LẦN TOKEN ĐỔI

```

export default {
  source: ['tokens/**/*.tokens.json'],
  platforms: {
    css: { transformGroup: 'css', buildPath: 'dist/css/',
      files: [{ destination: 'variables.css', format: 'css/variables' }] },
    ts: { transformGroup: 'js', buildPath: 'dist/ts/',
      files: [{ destination: 'tokens.ts', format: 'javascript/es6' }] },
    compose: { transformGroup: 'compose', buildPath: 'dist/android/',
      files: [{ destination: 'Tokens.kt', format: 'compose/object' }] }
  }
};

```

3.2 • Nâng cấp bước 2: thêm lớp “AI-readable”

SD code phải kèm ba thứ: **(a)** Storybook với stories đầy đủ + Storybook MCP để agent query props/usage thật thay vì đoán; **(b)** Code Connect mapping; **(c)** file rules cho agent.

× TRÍCH NGUỒN – Storybook (chính thức) + LogRocket

*Storybook MCP biến bộ Storybook sẵn có của bạn thành một nguồn dữ liệu có cấu trúc để AI agent tra cứu theo thời gian thực: trong codebase đang có những component nào, mỗi cái nhận props gì, dùng ra sao — kèm stories và tài liệu, ở định dạng tiết kiệm token. Nhờ được đọc tài liệu thật thay vì phải đoán, agent **hết “bịa” ra component không tồn tại** — vốn là dạng nợ code phổ biến nhất khi làm frontend cùng AI.*

github.com/storybookjs/mcp · [blog.logrocket.com](https://blog.logrocket.com/storybook-mcp-for-ai-aware-component-libraries) → Storybook MCP for AI-aware component libraries

BUTTON.FIGMA.TSX – CODE CONNECT: DẠY AI NODE FIGMA NÀY = COMPONENT NÀO

```
import figma from '@figma/code-connect';
import { Button } from '@components/ui/button';

figma.connect(Button, 'https://figma.com/design/xxx?node-id=123:456', {
  props: {
    variant: figma.enum('Variant', {
      Primary: 'default', Secondary: 'secondary', Ghost: 'ghost',
    }),
    size: figma.enum('Size', { Small: 'sm', Medium: 'default', Large: 'lg' }),
    label: figma.string('Label'),
  },
  example: ({ variant, size, label }) => (
    <Button variant={variant} size={size}>{label}</Button>
  ),
});
```

CLAUDE.MD (TRÍCH) – LUẬT CHO AGENT KHI SINH UI

```
## UI generation rules
- ONLY use components from `@/components/ui` – never write raw styled divs
  for patterns that exist (Button, Card, DataTable, FormField...).
- NEVER hardcode colors/spacing/font sizes. Use tokens:
  `var(--color-action-primary-bg)`, `theme.spacing.md`.
- Every form uses `<FormField>` + zod schema. Every table uses `<DataTable>`.
- After generating a screen: run `npm run lint:tokens && npm run test:ally`.
```

3.3 • Nâng cấp bước 3: phân tầng fidelity

Như phân tích ở mục 2.3 — hi-fi Figma chỉ cho flow mới/quan trọng; màn hình pattern chuẩn đi thẳng từ spec nghiệp vụ → AI compose → review trên bản chạy thật.

3.4 • Nâng cấp bước 4: thêm công kiểm chứng máy

Ví dụ Gate B — cấm giá trị hardcoded bằng lint; Gate C là visual regression (Playwright/Chromatic) so screenshot với baseline từ design:

.STYLELINTRC – CHẶN HEX/PX TRẦN TRONG CODE UI

```
{
  "rules": {
    "color-no-hex": true,
    "declaration-property-value-allowed-list": {
```

```
"/^(padding|margin|gap)/": ["/^var\\(--spacing/", "/^0$/"]
}
}
}
```

× TRÍCH NGUỒN – arXiv, nghiên cứu multi-agent frontend enterprise

*Các nghiên cứu về framework đa agent làm frontend “từ pixel đến production” cho doanh nghiệp có chung một kết luận: muốn vượt qua ngưỡng “prototype” để đạt chất lượng production, **vòng kiểm chứng tự động (verification loop) không phải là tính năng tùy chọn, mà là thành phần bắt buộc của kiến trúc.***

[arxiv.org](#) → [Beyond Prototyping: Autonomous, Enterprise-Grade Frontend Development via a Specialized Multi-Agent Framework](#)

04 Quy trình cộng đồng đề xuất mới nhất: “Design System as AI Context” (grounded generation)

Nếu phải chọn **một** quy trình có tên tuổi, có phương pháp luận, có sự ủng hộ rộng và cách triển khai rõ ràng nhất hiện nay, đó là workflow mà Figma chính thức mô tả trong bài “Design Systems And AI: Why MCP Servers Are The Unlock”, cộng hưởng với registry pattern của shadcn. Cộng đồng gọi nó là **grounded design-to-code** — AI không “dịch” thiết kế, mà **lắp ráp từ hệ thống đã được ground**.

× TRÍCH NGUỒN – Figma Blog: triết lý “flywheel” + 3 use case khuyến nghị

*AI giúp design system của bạn mạnh lên, và design system mạnh lại giúp AI sinh code tốt hơn — một vòng xoáy tự cường. Figma khuyến nghị ba cách khai thác: **(1)** để agent sinh component mới theo đúng pattern sẵn có, bằng cách kết hợp ngữ cảnh MCP từ thiết kế với code component hiện hữu; **(2)** để agent tự động hoá công việc token — tìm những chỗ còn thiếu token, đảm bảo token mới tuân thủ chuẩn đã định; **(3)** để agent làm “kiểm toán viên” — rà soát và gấn cò độ lệch giữa file Figma và codebase.*

“AI strengthens your design system, which powers better AI code generation.”

[figma.com/blog](#) → [Design Systems And AI: Why MCP Servers Are The Unlock](#)

4.1 · Năm giai đoạn triển khai

- 01 TOKEN HOÁ.** Export Figma Variables → DTCG JSON, đưa vào git, dựng Style Dictionary trong CI. Thời lượng thực tế: 1–2 tuần với DS sẵn có.
- 02 GROUND.** Viết Code Connect cho ~20 component tần suất cao nhất (Pareto: 20 component phủ ~80% màn hình); bật Storybook MCP; viết CLAUDE.md / cursor rules. Với React: cân nhắc xuất bản DS nội bộ dạng shadow registry — phân phối components, tokens, conventions và *instructions cho AI agents* về một mối.
- 03 GENERATE.** Quy trình hằng ngày: designer share link frame → agent (Claude Code/Cursor có Figma MCP) đọc structured context → compose màn hình từ component thật, không tái tạo pixel.
- 04 VERIFY.** Bốn gate tự động (build · token lint · screenshot diff · a11y) + human review PR. Đây là khác biệt giữa demo và sản phẩm thật.
- 05 LEARN.** Mỗi lần dev phải sửa tay output của AI = tín hiệu thiếu rule / thiếu mapping / thiếu token — cập nhật lại hệ thống, không sửa chay. Audit drift Figma ↔ code định kỳ (chính agent làm được qua MCP).

4.2 · Vì sao quy trình này thắng các phương án khác

SO VỚI	GROUNDING THẮNG VÌ
Screenshot-to-code (Anima, Locofy đời đầu)	Giữ 100% ngữ nghĩa thiết kế thay vì mất ~80% metadata khi export hình.
“Bỏ Figma, prompt thắng”	Giữ không gian exploration của designer (luận điểm Karri Saarinen) – Figma là nơi <i>search</i> , code là nơi <i>ship</i> ; đúng tinh thần “false dichotomy”.

SO VỚI	GROUNDED GENERATION THẮNG VÌ
Quy trình 4 bước nguyên trạng	Khác đúng 3 thứ: <i>một source of truth</i> (tokens + code) · có đường về (learn loop) · có cổng kiểm chứng máy.

05 Bốn khuyến nghị thực chiến

- Đừng bắt đầu bằng việc dựng DS Figma thật đẹp.** Bắt đầu bằng tokens + 20 coded components + Code Connect. Đó là 20% công sức tạo 80% chất lượng output AI.
- Đo drift như đo bug.** Chạy audit Figma ↔ code hàng tuần (chính agent làm được việc này qua MCP). Drift là thứ giết chết quy trình trong im lặng.
- AI không thay bước nào trong 4 bước — nó thay chất keo giữa các bước.** Các bước vẫn tồn tại; thứ biến mất là công việc “dịch tay” giữa chúng. Như Brad Frost dẫn lời Josh Clark: *“The design system carries the burden of the boring”* — và giờ AI gánh nốt phần boring của việc vận hành design system.
- Con người giữ 3 việc không nhượng:** quyết định pattern mới (design là search) · review trước merge · phản xử “component hay recipe” — đúng như Frost nhấn mạnh, đó vẫn là human judgment.

06 Tổ chức thực thi cho mô hình hai phòng

Lộ trình cho hai đơn vị — phòng Thiết kế và phòng Phát triển phần mềm — cùng xây dựng quy trình đến khi đủ điều kiện áp dụng ở production. Khung thời gian tính cho team 4–8 người mỗi phòng, design system cỡ vừa (~30–50 component).

6.1 · Ba nguyên tắc tổ chức trước khi chia việc

- Chỉ định 2 người làm cầu nối (liaison):** 1 designer thạo variables / component properties và 1 dev thạo kiến trúc FE. Mọi quyết định chung (naming, token, mapping) đi qua cặp này — không hợp cả hai phòng cho từng chi tiết. Brad Frost nhấn mạnh

design system thất bại chủ yếu vì đứt gãy giao tiếp giữa các phòng, không phải vì kỹ thuật.

2. **“Done” của một component được định nghĩa chéo:** chỉ được tính hoàn thành khi đủ cả 4 mảnh — Figma component chuẩn + code + Storybook story + Code Connect mapping. Thiếu một mảnh là chưa done, dù phòng nào đã xong phần mình.
3. **Không chờ hoàn hảo mới pilot:** giai đoạn 3 chạy pilot ngay với 20 component đầu tiên, không đợi đủ 50.

LỘ TRÌNH 6 GIAI ĐOẠN – THỨ TỰ PHỤ THUỘC

GĐ0 • Thống nhất nền móng (tuần 1-2)

hai phòng cùng làm – chốt source of truth, glossary, danh sách 20 component ưu tiên



GĐ1 • Token hoá (tuần 2-4)

Figma Variables 2 tầng ↔ repo DTCG ↔ Style Dictionary CI



GĐ2 • Component hoá (tuần 4-9)

hai phòng chạy song song theo lô 5 component, so khớp cuối mỗi lô



GĐ3 • Grounding AI + Pilot (tuần 9-11)

MCP + rules + pilot 3 màn hình thật, đo 3 chỉ số



GĐ4 • Verification gates + production (tuần 11-13)

4 gate vào CI · designer review trên staging · 2 sprint thật



GĐ5 • Vận hành (liên tục)

hội đồng DS 2 người · drift audit hàng tuần · learn loop

6.2 • Giai đoạn 0 – Thống nhất nền móng (tuần 1-2, hai phòng cùng làm)

#	NHIỆM VỤ	CHỦ TRÌ	ĐẦU RA NGHIỆM THU
0.1	Audit hiện trạng file Figma: % component có auto-layout, % giá trị đã gán variables, chất lượng naming	Thiết kế	Báo cáo audit + danh sách nợ cần trả
0.2	Audit codebase FE: framework, component sẵn có, mức độ hardcode màu/spacing	Phát triển	Báo cáo audit + quyết định stack (giữ / refactor)
0.3	Quyết định source of truth: tokens + code là chân lý, Figma là môi trường thao tác – chốt bằng văn bản	Cả hai (liaison)	1 trang quy ước, lãnh đạo hai phòng ký
0.4	Bảng thuật ngữ chung (glossary): tên component, props, variant – <i>một tên dùng ở cả Figma lẫn code</i>	Cả hai	Glossary lưu trong repo
0.5	Chọn 20 component ưu tiên theo tần suất xuất hiện trên màn hình thực tế (quy tắc Pareto)	Cả hai	Danh sách 20, có thứ tự

Điểm chặn: không sang GĐ1 nếu mục 0.3 chưa được chốt — mọi tranh cãi về sau đều quy về câu hỏi này.

6.3 • Giai đoạn 1 – Token hoá (tuần 2-4)

#	NHIỆM VỤ	PHÒNG	GHI CHÚ
1.1	Chuẩn hoá Figma Variables theo 2 tầng: primitive (<code>blue-500</code>) → semantic (<code>action/primary-bg</code>); xoá style rời rạc	Thiết kế	Designer chỉ dùng tầng semantic khi thiết kế màn hình
1.2	Dựng repo <code>design-tokens</code> : export DTCG JSON, cấu trúc thư mục, quy ước đặt tên	Phát triển	Theo chuẩn W3C DTCG 2025.10

#	NHIỆM VỤ	PHÒNG	GHI CHÚ
1.3	Dựng Style Dictionary trong CI: token đổi → tự build CSS vars / TS types / Kotlin / Swift → publish package	Phát triển	Có version + changelog tự động
1.4	Thiết lập sync hai chiều Figma Variables ↔ repo (Tokens Studio hoặc native DTCTG export)	Cả hai (liaison)	Test: đổi 1 màu ở repo → thấy ở Figma và ở app
1.5	Review chéo bộ token: designer duyệt ngữ nghĩa, dev duyệt tính khả dụng đa nền tảng	Cả hai	–

Nghiem thu GD1: đổi một token duy nhất → cả file Figma lẫn app build đều đổi theo, không ai phải sửa tay chỗ nào.

6.4 • Giai đoạn 2 – Component hoá song song (tuần 4-9)

Hai phòng chạy **song song theo từng lô 5 component**, so khớp cuối mỗi lô.

Phòng Thiết kế — với mỗi component:

- Dựng / chuẩn hoá trên Figma: auto-layout 100%, mọi giá trị gắn semantic token, không còn hardcoded.
- Component properties đặt tên **trùng props code** theo glossary (`variant` , `size` , `disabled` ...).
- Viết annotation hành vi ngay trong file: hover, focus, responsive, empty/error state.
- Dựng trang tài liệu sử dụng (khi nào dùng cái này thay cái kia).

Phòng Phát triển — với mỗi component:

- Code component đủ states, responsive, keyboard / a11y.
- Viết Storybook stories phủ mọi variant — đây là “tài liệu cho AI đọc”, không được bỏ qua.
- Unit test + a11y test (axe).

- Viết **Code Connect file** map node Figma ↔ component code (làm cùng liaison design).

Chung, cuối mỗi lô 5 component: phiên “so khớp” 1 giờ — đặt Figma và Storybook cạnh nhau, đối chiếu từng variant, ký xác nhận đủ 4 mảnh.

Nghiệm thu GD2: 20/20 component đạt định nghĩa “done” 4 mảnh; bảng theo dõi tiến độ không còn ô trống.

6.5 • Giai đoạn 3 – Grounding AI + Pilot (tuần 9-11)

#	NHIỆM VỤ	PHÒNG
3.1	Cài đặt hạ tầng agent: Figma MCP server, Storybook MCP, cấp quyền truy cập	Phát triển
3.2	Viết bộ rules cho agent (CLAUDE.md / cursor rules): chỉ dùng component từ thư viện, cấm hardcode, quy ước form/table, lệnh kiểm tra sau khi sinh	Phát triển (liaison design review)
3.3	Viết “checklist vệ sinh file” cho designer: điều kiện một frame được coi là sẵn sàng bàn giao (đã dùng component + variables + annotation)	Thiết kế
3.4	Dựng template file Figma cho màn hình nghiệp vụ: khung layout chuẩn, vùng ghi chú spec	Thiết kế
3.5	Pilot 3 màn hình thật (1 CRUD đơn giản, 1 form phức tạp, 1 màn có pattern mới): designer bàn giao link frame → dev dùng agent sinh code → đo đạc	Cả hai
3.6	Đo pilot bằng 3 chỉ số: thời gian design-bàn-giao → PR; % dòng code dùng component thư viện; số lần dev phải sửa tay output	Cả hai

Nghiệm thu GD3: màn CRUD đạt ≥80% code từ component thư viện và thời gian tạo màn hình giảm rõ rệt so với baseline. Nếu không đạt — nguyên nhân hầu như luôn nằm ở GD1/GD2 (token thiếu, mapping thiếu, file Figma bẩn): quay lại và trước khi mở rộng.

6.6 • Giai đoạn 4 – Verification gates + đưa vào production (tuần 11-13)

#	NHIỆM VỤ	PHÒNG
4.1	Gate B vào CI: stylelint/ESLint chặn hex, px trần – build đỏ nếu vi phạm	Phát triển
4.2	Gate C: visual regression (Playwright/Chromatic) – baseline từ design, PR nào lệch phải giải trình	Phát triển
4.3	Gate D: axe a11y scan trong CI	Phát triển
4.4	Quy trình review PR mới: designer review trên staging/preview deployment , không duyệt bằng cách nhìn Figma nữa	Thiết kế
4.5	Chạy 2 sprint thật với quy trình đầy đủ trên tính năng production thực	Cả hai

Tiêu chí “đủ điều kiện production”: ① 4 gates chạy tự động và đã chặn được ít nhất 1 vi phạm thật; ② 2 sprint liên tiếp không có hotfix nào do UI sai design; ③ designer xác nhận review trên staging thay thế được review mockup.

6.7 • Giai đoạn 5 – Vận hành (liên tục)

- **Hội đồng DS 2 người** (chính là cặp liaison): duyệt component mới, phán xử “component hay recipe”, giữ glossary.
- **Drift audit hàng tuần** — giao cho chính agent chạy qua MCP: so Figma ↔ code, báo cáo lệch.
- **Learn loop:** mỗi lần dev sửa tay output của AI → ticket phân loại (thiếu token / thiếu mapping / thiếu rule) → vá vào hệ thống trong tuần.
- **Mở rộng dần:** thêm component theo lô 5, luôn giữ định nghĩa done 4 mảnh; màn hình mới về pattern thì designer vẫn explore hi-fi trên Figma trước — không ép mọi thứ qua AI.

6.8 • Phân vai tổng thể (RACI rút gọn)

HẠNG MỤC	PHÒNG THIẾT KẾ	PHÒNG PHÁT TRIỂN
Token: ngữ nghĩa, giá trị	R (chịu trách nhiệm)	C (góp ý)
Token: pipeline, publish	C	R
Component Figma + annotation	R	C
Component code + Storybook + test	C	R
Code Connect mapping	C	R (làm cùng liaison design)
Rules cho AI agent	C	R
Review PR giao diện trên staging	R	C
Quyết định pattern/component mới	Đồng quyết qua hội đồng DS	Đồng quyết

6.9 • Hai cái bẫy phổ biến nhất của mô hình hai phòng

1. **Phòng thiết kế làm xong DS Figma rồi “ném qua tường”** — vì vậy mới cần định nghĩa done 4 mảnh và phiên so khớp cuối mỗi lô: không có khái niệm “phần tôi xong rồi”.
2. **Phòng phát triển coi verification gates là việc làm sau** — trong khi không có gates thì output của AI sẽ làm bản codebase nhanh hơn tốc độ hai phòng xây chuẩn.

07 Phụ lục: danh mục nguồn

CHÍNH THỨC [Design Systems And AI: Why MCP Servers Are The Unlock – Figma Blog](#)

ESSAY [Design systems in the time of AI – Brad Frost](#)

CHUẨN [Design Tokens Specification 2025.10 stable – W3C DTCG](#)

KỸ THUẬT [DTCG Design Tokens in Figma – Mykhaylo Rychkin](#)

HƯỚNG DẪN [How to structure Figma files for MCP – LogRocket](#)

HƯỚNG DẪN [Figma MCP Server: The CTO guide to design-to-code in 2026 – Alex Bobes](#)

REPO [storybookjs/mcp – GitHub](#)

KỸ THUẬT [Storybook MCP for AI-aware component libraries – LogRocket](#)

REPO/DOCS [shadcn/ui docs & registry · What is shadcn/ui – Vercel](#)

QUAN ĐIỂM [The most important Design System in 2026 that designers missed – Pawel Klasa](#)

PHÂN TÍCH [AI Design-to-Code Tools: The Complete Guide – Banani](#)

PHẢN BIỆN [Design-to-Code AI Is Not Magic – Shahid Pattani](#)

KHẢO SÁT [Developers still don't trust AI-generated code – CIO](#)

NGHIÊN CỨU [Beyond Prototyping \(multi-agent frontend\) – arXiv · UI-Bench – arXiv](#)

BÁO CÁO [Craft – AI in Design Report 2026, Designer Fund](#)

QUAN ĐIỂM [Design Is Search – Karri Saarinen, Linear · Linear UI update thread – X](#)

PHỎNG VẤN [Figma's CEO: design vs code là "false dichotomy" – Department of Product](#)

QUAN ĐIỂM [Why I stopped prototyping in Figma – ADPList Substack](#)
